🆕

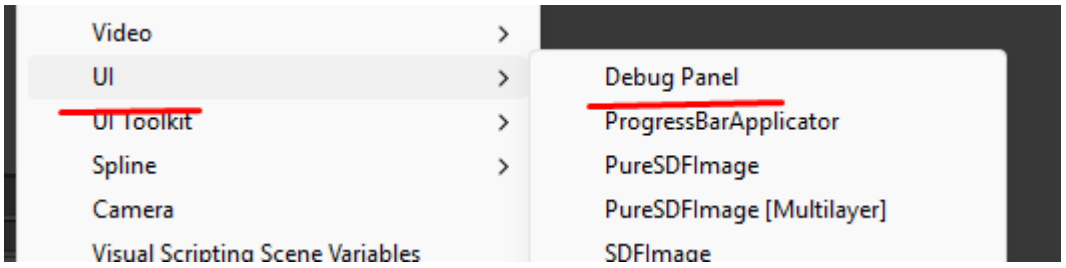
Getting Started

Added in version	1.0.3
Status	Up-to date

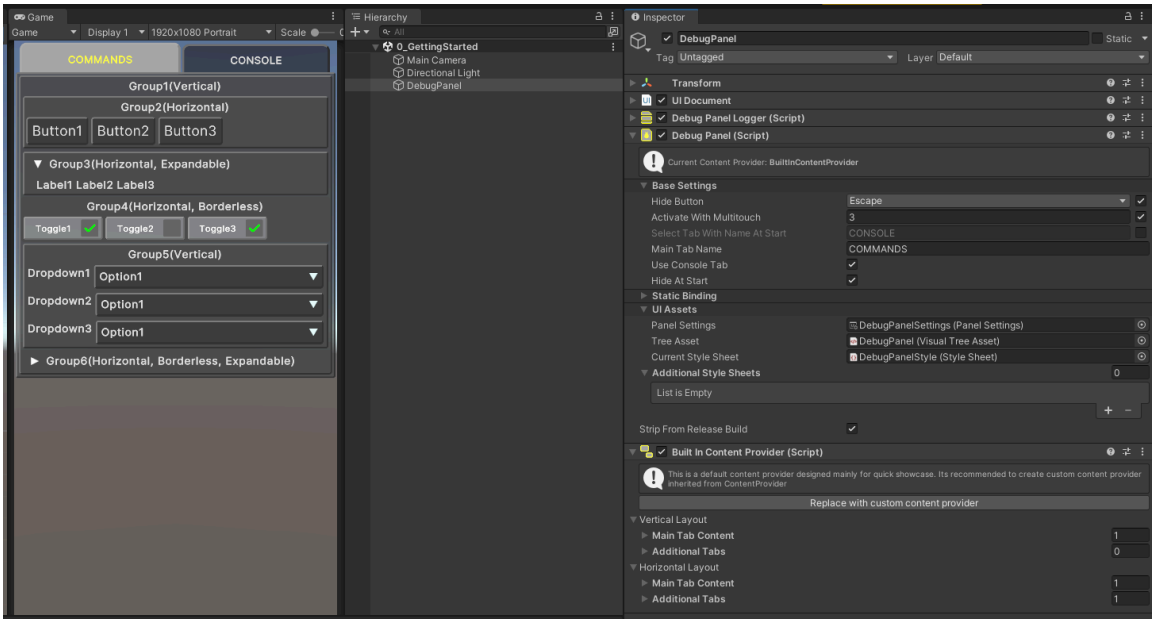
How to create Debug Panel
How debug panel works?
- Built In Content Provider
- Custom content provider
  - Code input
  - UI result
- UIDocument
- Activation and Base Settings
- Console Tab
- Static binding
- Strip from release build

How to create Debug Panel

1. As any other UI component with context menu UI/Debug Panel



2. You'll get game object called Debug Panel in your scene, with several component attached (will talk about them in detail later on)



3. PROFIT!!!

How debug panel works?

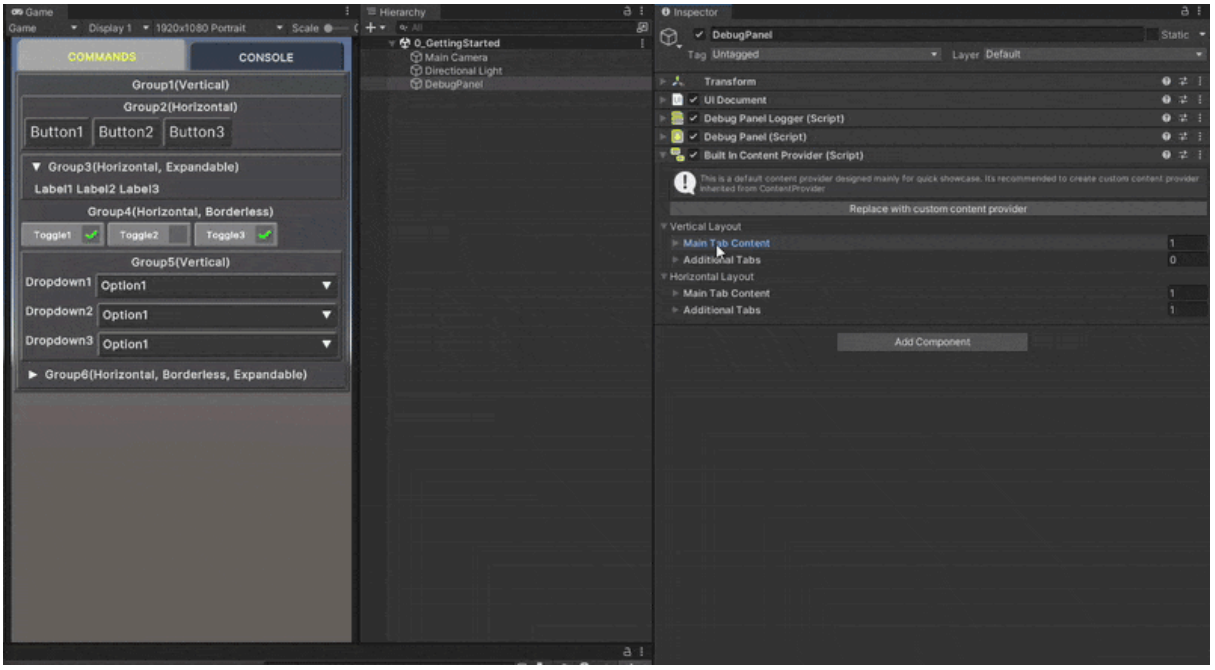
Debug panel is the UIToolkit based solution for cheats in games, best suited for mobile ones.

It allows to define in code all of your cheat buttons and get dynamic UI instantly.

Built In Content Provider

On gif below showcased Built In Content Provider, base component that exist just to showcase panel capabilities.

All the content in it defined in a serialized structures allowing to you to make changes and see how debug panel reflects them immediately even in edit mode.



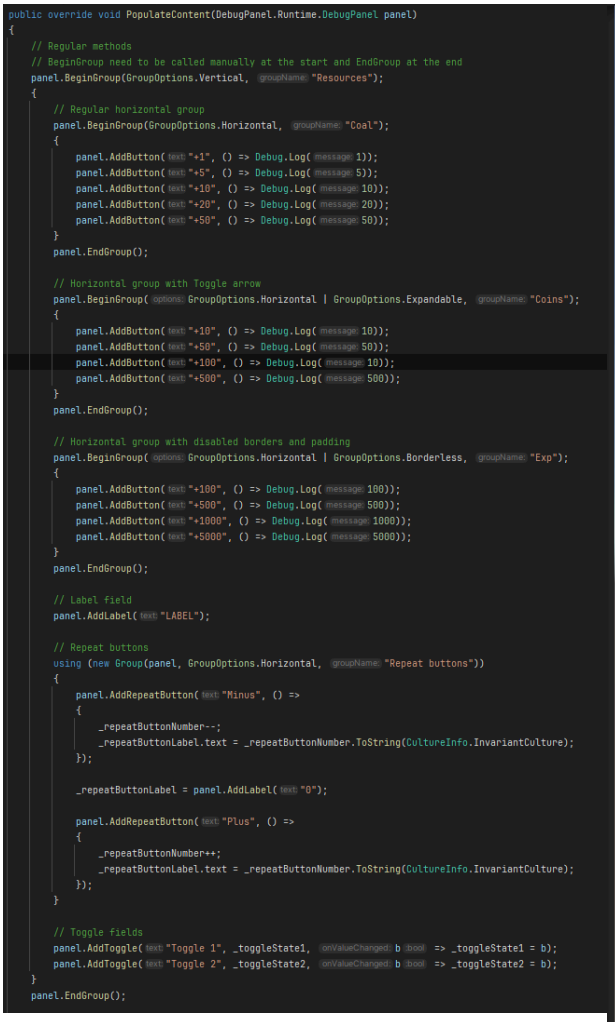
Built In Content Provider

Custom content provider

However the intended use is to make a custom content provider that defines all the element through code, and links them with your game systems.

Code input

UI result



```
public override void PopulateContent(DebugPanel.Runtime.DebugPanel panel)
{
    // Regular methods
    // BeginGroup need to be called manually at the start and EndGroup at the end
    panel.BeginGroup(GroupOptions.Vertical, groupName: "Resources");
    {
        // Regular horizontal group
        panel.BeginGroup(GroupOptions.Horizontal, groupName: "Coal");
        {
            panel.AddButton( text: "+1", () => Debug.Log( message: 1));
            panel.AddButton( text: "+5", () => Debug.Log( message: 5));
            panel.AddButton( text: "+10", () => Debug.Log( message: 10));
            panel.AddButton( text: "+20", () => Debug.Log( message: 20));
            panel.AddButton( text: "+50", () => Debug.Log( message: 50));
        }
        panel.EndGroup();

        // Horizontal group with Toggle arrow
        panel.BeginGroup( options: GroupOptions.Horizontal | GroupOptions.Expandable, groupName: "Coins");
        {
            panel.AddButton( text: "+10", () => Debug.Log( message: 10));
            panel.AddButton( text: "+50", () => Debug.Log( message: 50));
            panel.AddButton( text: "+100", () => Debug.Log( message: 10));
            panel.AddButton( text: "+500", () => Debug.Log( message: 500));
        }
        panel.EndGroup();

        // Horizontal group with disabled borders and padding
        panel.BeginGroup( options: GroupOptions.Horizontal | GroupOptions.Borderless, groupName: "Exp");
        {
            panel.AddButton( text: "+100", () => Debug.Log( message: 100));
            panel.AddButton( text: "+500", () => Debug.Log( message: 500));
            panel.AddButton( text: "+1000", () => Debug.Log( message: 1000));
            panel.AddButton( text: "+5000", () => Debug.Log( message: 5000));
        }
        panel.EndGroup();

        // Label field
        panel.AddLabel( text: "LABEL");

        // Repeat buttons
        using (new Group(panel, GroupOptions.Horizontal, groupName: "Repeat buttons"))
        {
            panel.AddRepeatButton( text: "Minus", () =>
            {
                _repeatButtonNumber--;
                _repeatButtonLabel.text = _repeatButtonNumber.ToString(CultureInfo.InvariantCulture);
            });

            _repeatButtonLabel = panel.AddLabel( text: "0");

            panel.AddRepeatButton( text: "Plus", () =>
            {
                _repeatButtonNumber++;
                _repeatButtonLabel.text = _repeatButtonNumber.ToString(CultureInfo.InvariantCulture);
            });
        }

        // Toggle fields
        panel.AddToggle( text: "Toggle 1", _toggleState1, onValueChanged: b /*bool*/ => _toggleState1 = b);
        panel.AddToggle( text: "Toggle 2", _toggleState2, onValueChanged: b /*bool*/ => _toggleState2 = b);
    }
    panel.EndGroup();
```

Part of the code that produces UI



Final UI result

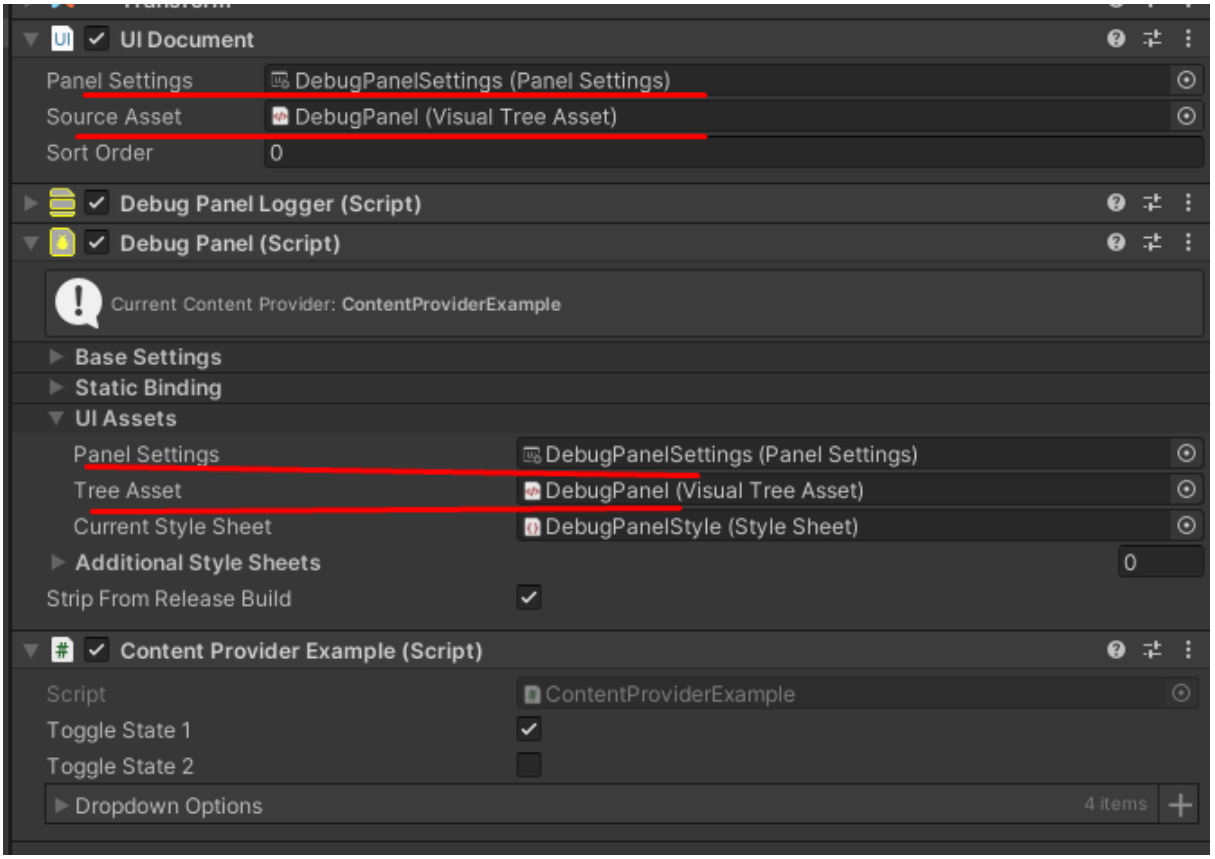
Above is the example of how buttons/groups/toggles/dropdowns/labels defined in code, and how UI is generated for them.

> 💡 Will go over custom content provider in first sample ≡ Simple Content Provider

UIDocument

UIDocument is the component that renders UIToolkit in runtime.

Debug panel injects two assets needed for its work, being the Panel Settings and Source Asset

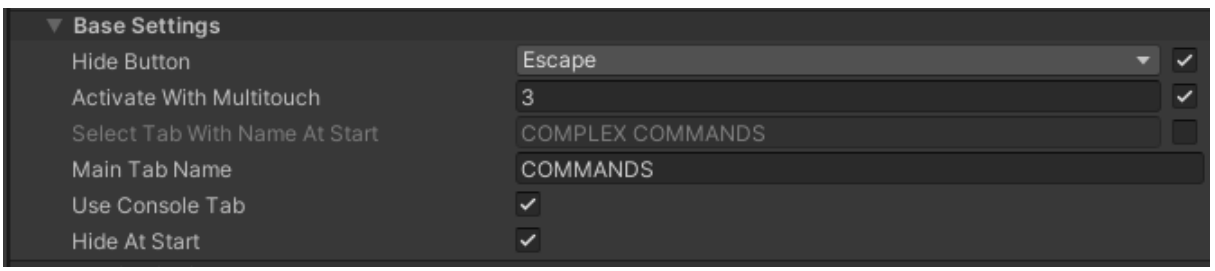


Debug panel injects default tree assets and panel settings to UIDocument

> ⚠️ Debug Panel controls UIDocument, keep that in mind

Provided assets is the default ones, and there is mostly no use in overriding them. However assigning different panel settings for your needs might be useful.

Activation and Base Settings



If `Hide At Start` property is enabled debug panel will be hidden at start, to show it Activation need to happen.

By default activation happens with the press of Escape key, or a multitouch with 3 fingers.

`Hide Button` - Escape keycode is escape on pc, back button on android, or other platforms like Amazon Fire TV (android based).

`Activate With Multitouch` - for mobile platforms with touch screen, might be convenient to open debug panel with triple touch

> ▶️ Activation can be customized, for example with button on UI, like done here: ⌨️ Activation Override

`Select Tab With Name At Start` - debug panel can have multiple tabs, this allows to select particular tab at startup. This also chooses tab in edit mode, for quick edits in this tab.

`Main Tab Name` - panel always contains at least one tab, determine its name here

`Use Console Tab` - enables , its optional

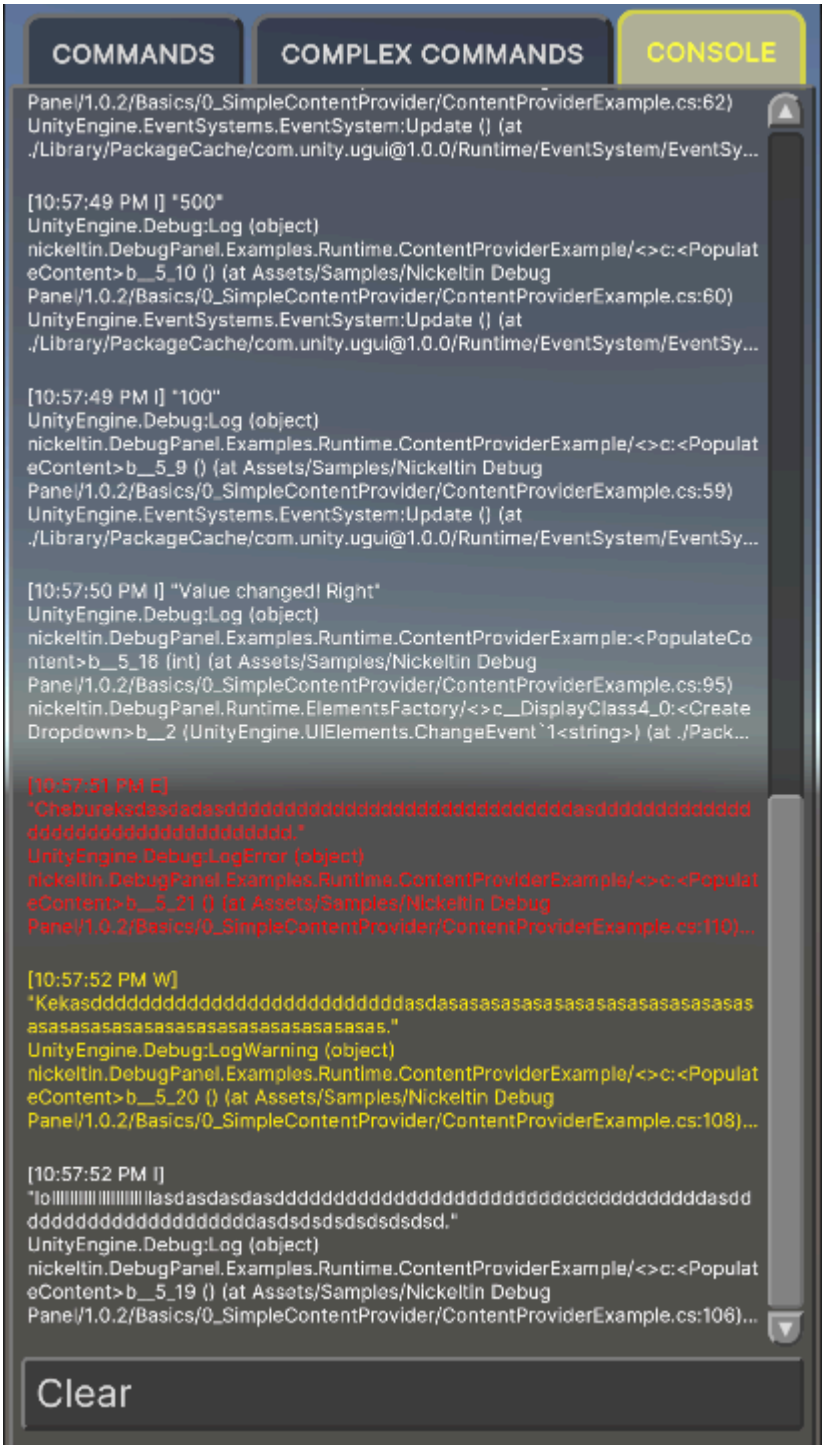
Console Tab

Console tab is optional tab that shows latest logs.

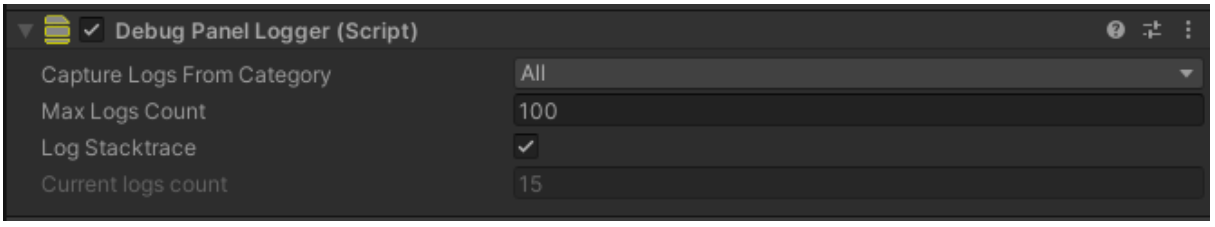
Logs is color coded depending on the type, Info/Warning/Error and can show stack trace.

Rich text is supported so you can format them the same as in editor, with different text colors, bold or italic.

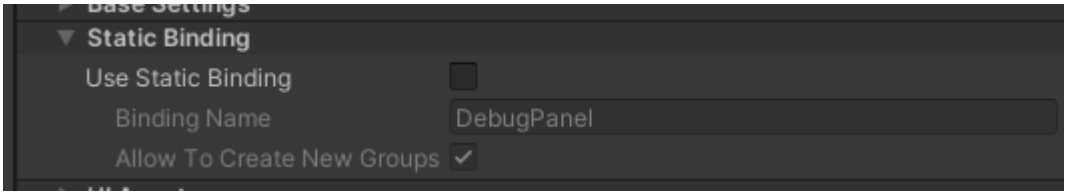
> ℹ️ If you click at log entry it will copy its content into system paste buffer



Logging settings can be adjusted in Debug Panel Loger component



Static binding



Static binding is process of populating debug panel content bypassing the content provider.

It scans scans codebase for methods with `[DebugPanelButtonAttribute]` and creates corresponding buttons for them.

> 💡 More about it in dedicated section 🔗 Static Binding

Strip from release build

Pretty simple setting, do don't want your cheats to be included in release application, right?😅

If this toggle enabled at build time DebugPanel game object attached to will be stripped for all of the scenes.

> ⚠️ Keep in mind since game object is removed in scene, all of it's childs will be gone as well.
> Also if debug panel is not in scenes, but you rather spawn it from prefab somewhere it will still spawn, and you need to make release build checks yourself.

At ideal conditions if debug panel in your persistent scene, and build stripping is enabled, since it's not referenced anywhere else none of its code or assets should go into the release build.

> 👏 **Congrats on creating first working Debug Panel and figuring out its basic settings!**